

Live Multi-View Surgical Workflow Segmentation

Technical Architecture Report | Candidate Submission | July 2026

Purpose. Demonstrate a clean, reproducible pipeline for live surgical phase segmentation from one to three camera streams, while addressing privacy, annotation cost, occlusion, cloud scale, and product-facing quality.

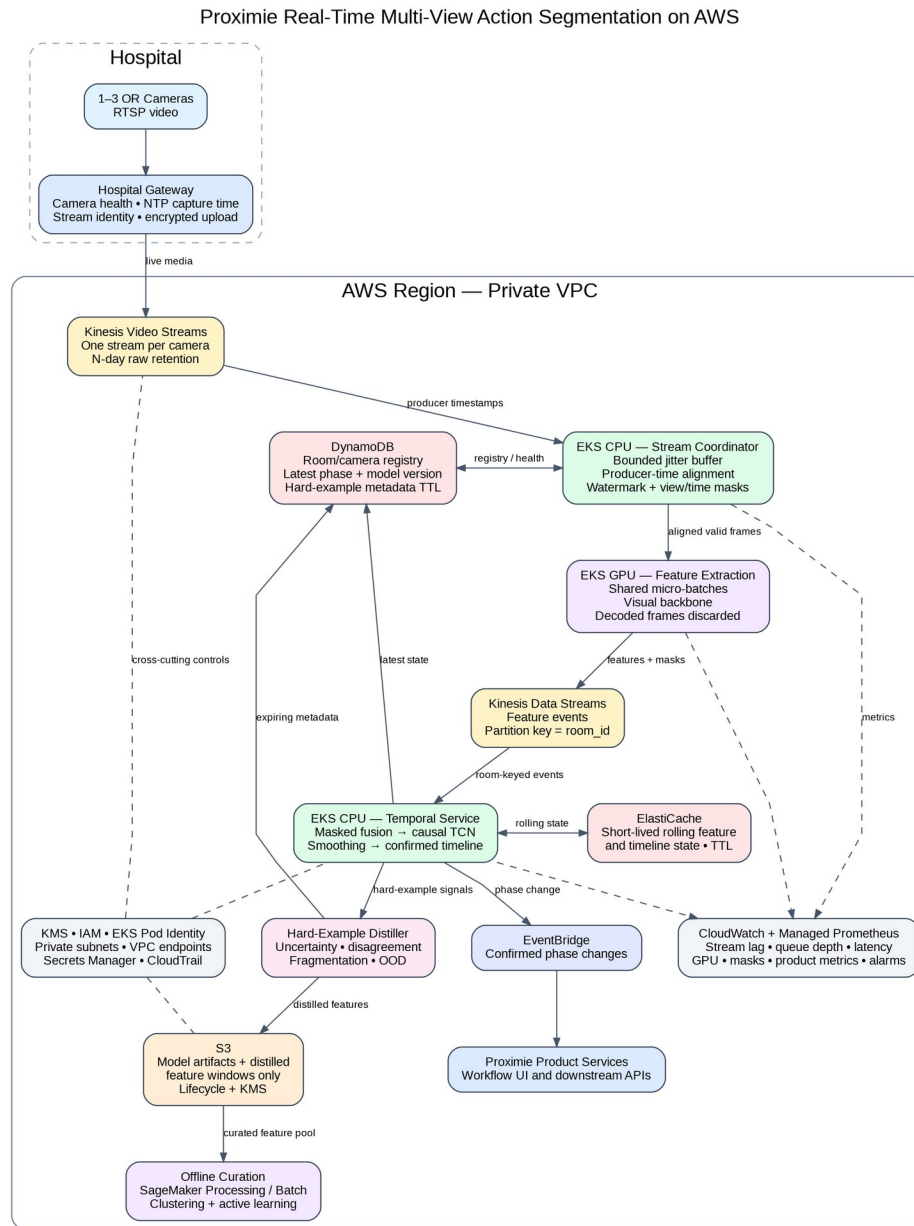


Figure 1. Proposed AWS production architecture for synchronized multi-view ingestion, feature extraction, temporal inference, hard-example distillation, and downstream workflow events.

1-3
camera views

1 Hz
prototype feature rate

61 s
causal receptive field

103k
trainable parameters

1. Scope and Design Choices

This prototype focuses on the temporal part of the system. It accepts precomputed visual features instead of raw video, supports one to three synchronized views, predicts one workflow phase per second, and turns noisy predictions into a contiguous timeline. The five mock phases are Empty, Patient Present, Preparation, Operation, and Closing.

The design stays deliberately small. Masked mean fusion handles missing cameras. A causal Temporal Convolutional Network, or TCN, uses only current and past evidence. Four residual blocks with dilations 1, 2, 4, and 8 give each prediction a 61-second receptive field. A five-second causal average and an eight-second confirmation rule reduce short false phase switches.

The synthetic dataset makes the repository self-contained. It includes camera-specific bias, temporally correlated noise, Patient Present disturbances, ambiguous boundaries, temporary Operation occlusions, and complete camera dropout. These choices let the tests exercise the failure modes named in the challenge without claiming real clinical performance.

2. Data Strategy and Annotation Lifecycle

2.1 Hard examples from the live pipeline

The live service should flag difficult intervals while the raw stream is still inside its approved retention window.

Useful triggers include low confidence, high uncertainty, frequent phase changes, disagreement between cameras, unstable boundaries, raw-versus-cleaned disagreement, reduced view availability, and features that sit far from the training distribution.

I would store a distilled package instead of another permanent video copy. The package contains pseudonymous room and time identifiers, feature vectors around the event, view masks, capture-time skew, class probabilities, raw and cleaned predictions, the selection reason, and model versions. When policy permits, it may also include a temporary pointer to the original Kinesis Video Streams interval. That pointer naturally expires with the source video.

Feature vectors still need privacy controls. They may preserve information about people, equipment, room layout, or procedure context. I would encrypt them, restrict access, log every review, and expire them using the same governance process as other sensitive clinical data.

2.2 Active learning on historical video

The selection unit should be a short clip, not an isolated frame. Temporal labels need context before and after a possible transition. The offline curation pipeline first extracts low-rate features, then scores candidate clips using uncertainty, model disagreement, rare clusters, camera disagreement, boundary instability, and underrepresented hospitals or views.

Temporal deduplication is essential. Ten nearby clips often describe the same event. The pipeline should select one representative example and then enforce diversity across hospitals, rooms, camera positions, procedure durations, occlusion patterns, and transition types. This gives annotators a smaller batch with higher information value.

2.3 Reducing annotation cost

I would combine self-supervised pretraining, model-assisted labeling, and focused review. The visual encoder can first learn from unlabeled video using masked reconstruction, temporal order, and cross-view consistency. A teacher model then proposes phase timelines. Annotators correct boundaries instead of labeling every second from scratch, while high-confidence interior regions can receive provisional pseudo-labels.

Boundary guidance matters as much as the model. The annotation tool should show synchronized views, proposed phases, confidence over time, and camera availability. The labeling guide must define exactly when each phase starts and ends. Agreement should use a time tolerance, followed by clinical adjudication for disputed examples.

3. Modeling and Multi-Sensor Fusion

3.1 Timestamp alignment and missing views

Frame numbers are not a synchronization strategy. The gateway attaches a capture timestamp, room identifier, camera identifier, and sequence number. In AWS, a room-level coordinator keeps a small jitter buffer and builds a common inference grid. The initial design uses one feature timestamp per second, a two-second buffer, and nearest-frame matching within 250 milliseconds.

Missing evidence remains explicit. If two cameras arrive and the third does not, the system sets that camera's view mask to false and continues. If every view is missing, the time mask becomes false and the timeline reports unavailable. It never converts missing evidence into Empty.

3.2 Fusion baseline and production path

The prototype uses masked mean fusion because it is easy to inspect and hard to misuse. It averages only valid camera features, supports one to three views, and preserves a clean contract for the temporal model. Its main weakness is equal weighting. A heavily occluded but technically valid camera may contribute as much as the clearest view.

The next comparison should be reliability-weighted fusion. A small view-quality head can score blur, occlusion, and feature confidence before aggregation. Cross-view attention is another option, but I would only keep it if it improves occlusion recovery or boundary precision enough to justify the extra compute and operational complexity. Where calibration is stable, person and equipment detections can also be mapped into a shared room coordinate system.

3.3 Temporal model and temporal scale

The TCN is causal by construction. At second 400 it can use roughly seconds 340 through 400, but it cannot use second 401. This makes the prototype honest about live inference. The repository includes a causality test that changes all future features and verifies that earlier logits stay identical.

Patient Present and Operation need different temporal behavior. Patient Present benefits from long context and strong stability checks. Operation boundaries need faster response. The current global smoothing rule removes fragments but delays transitions. A production version should use class-specific confirmation thresholds or a two-branch model with a short local path for boundaries and a longer context path for stable workflow state.

3.4 Prototype evaluation

Metric	Raw model	Cleaned timeline
Frame accuracy	99.70%	93.72%
Macro F1	99.67%	93.04%
Mean edit score	95.09	100.00
Segmental F1@50	97.12%	96.04%
Patient Present extra fragments	3	0
Mean absolute boundary error	about 1 s	5.86 s

The result shows the intended trade-off. Raw predictions are highly accurate but contain extra segments. Cleanup restores the correct phase order and removes every extra Patient Present fragment, while adding a measurable confirmation delay. These synthetic numbers validate the pipeline and metric behavior. They do not estimate real surgical performance.

4. AWS Cloud Architecture

4.1 Live data path

Each camera writes to its own Amazon Kinesis Video Stream through the hospital gateway. Kinesis Video Streams acts as the governed temporary raw source, with retention set to the hospital-approved N-day limit. The design does not create a second permanent raw-video archive in S3.

An EKS CPU service coordinates the streams. It aligns producer timestamps, applies the jitter watermark, and emits synchronized valid frames with view masks. EKS GPU workers then create visual features in shared micro-batches and discard decoded frames after extraction.

Feature events move through Kinesis Data Streams using room_id as the partition key. This keeps room events ordered and separates GPU feature extraction from the CPU temporal service. The temporal service performs masked fusion, causal TCN inference, smoothing, and phase confirmation.

4.2 Stateful inference and product output

Temporal inference is stateful. Each room needs a rolling feature window, current confirmed phase, pending phase, confirmation count, last processed sequence, and model version. Active state stays in memory and is copied to ElastiCache with a short time to live so another pod can recover after a restart.

DynamoDB stores durable room configuration and the latest product state. Confirmed phase changes publish through EventBridge to workflow interfaces and downstream APIs. Hard-example metadata also lives in DynamoDB with expiration, while distilled feature windows and model artifacts use encrypted S3 with lifecycle rules.

4.3 Security, monitoring, and failure behavior

The workload runs inside private subnets with VPC endpoints. KMS encrypts data at rest. IAM roles use least privilege through EKS Pod Identity. Secrets Manager holds credentials, and CloudTrail records administrative access.

CloudWatch and Managed Prometheus track stream health, timestamp skew, late frames, feature lag, queue depth, p95 latency, available-view count, prediction confidence, phase-switch rate, unavailable time, and GPU memory pressure. The system keeps working when one camera fails, marks a timestamp unavailable when every camera fails, drops frames that arrive after the watermark, and restores temporal state after pod replacement. It should never invent a confident phase when evidence is absent.

5. Cost Strategy

Visual feature extraction dominates cost, so the design protects GPU time first. A lightweight occupancy or scene-change gate can reduce backbone calls in a stable empty room. Sampling can remain low during stable states and rise near a possible transition or during Operation. Periodic forced checks prevent the system from getting stuck in an incorrect state.

GPU workers share micro-batches across rooms. CPU services handle synchronization, fusion, TCN inference, timeline cleanup, and event creation. Autoscaling should react to queue depth, oldest waiting event, active cameras, and end-to-end latency, not GPU utilization alone. The live path keeps a stable On-Demand or reserved baseline, while offline curation and retraining can use Spot capacity.

Retention controls cost as well as privacy. Raw video expires in Kinesis Video Streams. Distilled hard-example windows expire through S3 lifecycle rules, and DynamoDB metadata expires through record-level time to live. The curation store should never grow without an explicit retention budget.

6. Product Metrics and Decision Rules

Frame accuracy alone can hide a poor timeline. The evaluation stack therefore reports frame accuracy, per-class precision and recall, macro F1, confusion matrices, edit score, segmental F1, and start and end boundary errors. Product metrics then focus on the two difficult phases.

Patient Present	Operation
• Extra fragments • False-positive duration • Missed duration • Switches per hour • Stable confirmation delay	• Start and end delay • Coverage ratio • Coverage during occlusion • Missed duration • Recovery after occlusion

Thresholds should follow the product use case. Retrospective analytics may accept several seconds of delay in exchange for a clean timeline. A live alert may need faster boundaries and tolerate more switching. I would tune class-specific thresholds with clinical and product stakeholders rather than select one global rule from frame accuracy alone.

7. Limitations and Next Steps

The prototype is a structural baseline, not a clinical benchmark. It uses synthetic features, a fixed five-phase order, fixed-length sequences, equal-weight fusion, and one post-processing policy. Real deployment work must cover variable procedure lengths, missing or repeated phases, aborted workflows, new room layouts, camera replacement, calibration drift, longer outages, and agreed clinical boundary definitions.

The next experiments are practical and measurable. First, compare masked mean against reliability-weighted fusion. Second, compare the current TCN against a multi-scale temporal model. Third, tune class-specific confirmation rules. Fourth, test adaptive sampling against fixed one-second features. Fifth, evaluate feature retention through a formal privacy review and confirm that the selected hard-example representation supports useful error analysis.

The repository is ready for those experiments. Ingestion, feature generation, synchronization masks, fusion, temporal modeling, timeline generation, metrics, and error analysis are separate modules with automated tests. The current submission runs end to end, saves the best checkpoint, reloads it, evaluates held-out sequences, exports timelines and metrics, and makes every deliberate simplification visible.

Bottom line. The design keeps raw video governed, uses multi-view evidence when available, preserves uncertainty when evidence disappears, measures both model quality and clinical workflow behavior, and scales the expensive visual stage independently from the lightweight temporal stage.